

When Graph Structure Becomes a Liability: A Comprehensive Benchmark of GNN Architectures for Temporal Fraud Detection on Bitcoin Transaction Networks

Saket Maganti^{a,*}

^a*School of Computer Science and Engineering, VIT-AP University, Amaravati, Andhra Pradesh 522237, India*

Abstract

Graph Neural Networks (GNNs) have become a popular choice for financial fraud detection, largely because transaction networks carry structural signals—fraud rings cluster densely, layering schemes form distinctive chains—that individual node features cannot capture alone. Yet this paper shows that the assumed advantage of GNNs does not hold once evaluation is conducted rigorously across time.

We benchmark four architectures—MLP, GCN, GraphSAGE, and GAT—against three class-imbalance strategies on the Elliptic Bitcoin Dataset (203,769 nodes, 234,355 edges, 49 temporal time steps), applying inductive mini-batch evaluation with three-seed statistical validation and supplementing the main comparison with thirteen targeted analyses. The headline finding is counterintuitive: a plain three-layer MLP that reads only node features achieves $F1 = 0.744 \pm 0.005$, beating every GNN. GraphSAGE, the strongest graph model, reaches $F1 = 0.697 \pm 0.009$ — a gap of 4.7 percentage points.

The explanation lies in temporal distribution shift. Between the training window (steps 1–34, 11.6 % illicit) and the late test window (steps 43–49), the fraud rate falls as low as 0.3 %—a 39-fold reduction. Every tested model collapses from $F1 \approx 0.95$ at step 35 to $F1 < 0.02$ after step 43. Crucially, the collapse is a *recall* failure: models stop predicting fraud almost entirely, with the predicted fraud rate dropping from 8.2 % to 1.7 %.

Random Forest feature attribution reveals that 82.4 % of discriminative power comes from local, transaction-level features that stay statistically stable across the temporal split. This directly explains why the MLP is competitive: it relies on exactly those stable features, while GNNs additionally learn topological patterns from early time steps that grow misleading as fraud behaviour evolves. Fraud community structure is real and large—fraud nodes show 54 % fraudulent neighbours versus 0.56 % for licit nodes, a $96\times$ ratio—but exploiting it backfires once the fraud distribution shifts. Temperature scaling and per-step threshold tuning both leave F1 unchanged ($\Delta F1 \approx 0$), ruling out probability miscalibration as the root cause. An EvolveGCN ablation on CPU ($F1 = 0.29$ versus published GPU result $F1 = 0.77$) establishes the hardware requirements for temporal architectures. A business cost analysis under asymmetric error penalties ($FN = \$50k$, $FP = \$2k$) shows that the highest-F1 configuration is not always the cheapest to operate.

Three practical conclusions follow: fraud detection systems require periodic retraining because static models cannot track rapid fraud-rate changes; graph structure introduces harmful inductive bias under distribution shift even when it is genuinely discriminative under stable conditions; and temporal GNNs need full snapshot sequences and GPU hardware to deliver their published advantages. Code and checkpoints are publicly available.

*Corresponding author.

Email address: saket.maganti@vitap.ac.in (Saket Maganti)

Keywords: Graph neural networks, Financial fraud detection, Temporal distribution shift, Elliptic Bitcoin Dataset, Class imbalance, GraphSAGE, Graph attention networks, Concept drift, Temperature scaling, Inductive learning

1. Introduction

1.1. Why Graph Structure Looks Attractive for Fraud Detection

Financial fraud costs the global economy well over \$5 trillion per year [1]. On blockchain networks the problem is particularly acute: Bitcoin transactions are pseudonymous, irreversible, and publicly visible, making them a favoured vehicle for ransomware payments, darknet commerce, and Ponzi schemes. Detection must happen before confirmations accumulate and funds move beyond reach.

The case for Graph Neural Networks (GNNs) is intuitive. Organised fraud does not operate through isolated transactions; it relies on networks. Money laundering requires layering across multiple wallets; fraud rings coordinate and share infrastructure; mixing services create topologically distinctive flow patterns. None of these signals are visible when transactions are modelled individually, but all of them should be recoverable by an architecture that aggregates information across the transaction graph.

The Elliptic Bitcoin Dataset [2], released in 2019, was designed precisely to test this hypothesis. It labels 46,564 Bitcoin transactions as illicit or licit, provides 165 node features per transaction, and orders the data across 49 temporal time steps, offering a realistic benchmark for graph-based fraud detection. The published literature has consistently rewarded GNN approaches: GCN (F1 = 0.70, Weber et al. [2]), augmented GCN (F1 = 0.74, Alarab et al. [3]), GraphSAGE with self-supervised pre-training (F1 = 0.75, Lo et al. [4]), and EvolveGCN (F1 = 0.77, Pareja et al. [5]) all outperform feature-only baselines reported in the same works, reinforcing the narrative that graph structure helps.

1.2. What Prior Work Overlooks

Looking more carefully at this literature, two methodological choices limit how confident we should be in the conclusions. First, every prior study uses *transductive* evaluation: the full graph, including test-time nodes and their edges, is present in memory during training. When a GNN evaluates a test node, it can propagate information directly through edges to labeled training neighbours, effectively leaking ground-truth labels forward from training into test inference. In a real deployment, future transactions do not exist at training time, so this setup is optimistic about what the model actually learns.

Second, and more consequentially, no prior study reports per-timestep performance across the 15 test time steps. All papers aggregate metrics over the entire test window. This obscures whether the model works throughout the test period or only during the first few steps. As we show, the answer is dramatically the latter: models are strong in steps 35–42 and virtually useless in steps 43–49.

When both issues are corrected, inductive evaluation and per-step analysis, a picture emerges that is quite different from the published literature.

1.3. What This Paper Contributes

We conduct a benchmark study built around four research questions.

RQ1 Which architecture–strategy combination performs best under inductive evaluation with statistical validation?

RQ2 How does performance vary across individual test time steps, and what drives the degradation in later steps?

RQ3 Do graph architectures provide discriminative benefit beyond node features alone, and if not, why not?

45 **RQ4** Can post-hoc calibration, temperature scaling or threshold tuning, recover performance lost to temporal shift?

Our contributions are:

- A statistically validated benchmark across 4 architectures $\times$ 3 imbalance strategies $\times$ 3 random seeds, all under inductive evaluation.
- 50 • The first systematic per-timestep analysis on Elliptic, revealing that aggregate test-set F1 hides near-complete failure in steps 43–49.
- Feature importance analysis (Random Forest) that quantifies why local features outperform structural aggregation under temporal shift.
- An impossibility result for post-hoc calibration: $\Delta F1=0$ for temperature scaling and
55 threshold tuning.
- An EvolveGCN hardware ablation that isolates CPU vs. GPU performance differences.
- Business cost analysis showing that the model with the highest F1 is not necessarily the most cost-effective under asymmetric penalties.
- All code and checkpoints: <https://github.com/Saket-Maganti/gnn-fraud>.

60 2. Related Work

2.1. GNN-Based Fraud Detection

The application of graph learning to transaction networks predates deep learning. Early work used belief propagation and Markov random fields to catch fraudulent accounts in payment graphs [6]. Differentiable message-passing architectures made it possible to learn
65 the aggregation function end-to-end, improving substantially over those earlier hand-crafted propagation rules.

On the Elliptic Dataset specifically, Weber et al. [2] provided the founding GCN baseline ($F1=0.70$), noting that the 71 manually engineered aggregate features in the dataset already provide a surrogate for one round of message passing, so the additional gain from GCN over
70 a well-equipped MLP is modest. Pareja et al. [5] introduced EvolveGCN, which updates GCN weight matrices via a GRU across time snapshots rather than fixing them once during training; on Elliptic this yielded $F1=0.77$ but only with full GPU-based training over all 34 training snapshots. Alarab et al. [3] combined GCN with feature engineering and class-weighted loss to achieve $F1=0.74$. Lo et al. [4] applied GraphSAGE with structural graph
75 augmentation and self-supervised pre-training, reaching $F1=0.75$.

None of these works include an inductive evaluation protocol or report per-timestep performance. The gap between what they measure and what a deployed model would experience is the central motivation for our study.

Outside the Elliptic Dataset, GNN fraud detection has been applied to e-commerce at
80 scale [7], financial services networks [8], and insurance claims [9]. A common thread is the assumption of distributional stability between training and deployment, which holds approximately in some settings but fails in ours.

2.2. Handling Class Imbalance

Fraud datasets are severely imbalanced: legitimate transactions comfortably outnumber
85 fraudulent ones by a factor of 7–10 $\times$ in Elliptic and often far more in production systems. Standard cross-entropy loss allocates most gradient to the majority class, leaving the model with little incentive to detect rare fraud.

Chawla et al. [10] proposed SMOTE, which synthesises minority samples by linearly
interpolating between existing minority examples in feature space. Zhao et al. [11] adapted
90 this to graph data by interpolating in embedding space and rewiring edges for synthetic nodes. Our approach is philosophically different: we clone structurally complete 1-hop ego-graphs around fraud seed nodes rather than interpolating, which preserves the dense intra-fraud connectivity that GNNs should learn to exploit.

Focal loss [12] down-weights easy-to-classify majority examples through a modulating
95 factor $(1-p_t)^\gamma$, concentrating gradient on the borderline cases where misclassification matters. Square-root class weighting, which we adopt, offers similar benefits with lower variance than linear weighting in our experiments.

2.3. Temporal Graph Learning

EvolveGCN [5] is the standard temporal GNN baseline for Elliptic. DySAT [13] and
100 TGAT [14] represent more recent temporal attention architectures. All require GPU hardware and access to full temporal sequences to operate as intended. Our EvolveGCN ablation quantifies the performance cost of departing from these conditions.

Concept drift, the statistical phenomenon where the target distribution changes over time
[15], is endemic to fraud detection. Fraudsters adapt continuously in response to detection,
105 prices shift, and law enforcement actions disrupt specific criminal ecosystems. The Elliptic test period captures exactly this kind of abrupt drift.

2.4. Calibration

Guo et al. [16] showed that modern neural networks are systematically overconfident and
proposed temperature scaling as a clean post-hoc fix: divide logits by scalar $T > 1$ before
110 softmax. The key property for our analysis is that temperature scaling is a *rank-preserving* transformation: it changes probability values but not their ordering, so the argmax prediction at a fixed threshold is unchanged. Temperature scaling can therefore only improve F1 by moving miscalibrated probabilities across the threshold, which we show does not happen during temporal collapse.

3. Dataset and Experimental Setup

3.1. The Elliptic Bitcoin Dataset

Elliptic, a blockchain analytics company, assembled this dataset from the Bitcoin trans-
action ledger and released it publicly in 2019 [2]. Each node represents a single Bitcoin
transaction. Directed edges run from the outputs of one transaction to the inputs of another,
120 encoding BTC flow; following prior work we treat them as undirected to enable bidirectional message passing. Statistics are given in Table 1.

Each of the 165 node features describes directly observable transaction properties. Fea-
tures 0–93 are *local*: inputs/outputs counts, transaction amounts, fees, timing, and statistical
aggregates thereof. Features 94–164 are *aggregate*: summaries of the local features of imme-
125 diate neighbours, serving as a manually computed proxy for a single round of GNN message
passing. This design means the MLP already has access to neighbourhood information via
fixed aggregate features; the GNN’s additional contribution is dynamic, learned propagation.

Table 1: Elliptic Bitcoin Dataset summary. The 7.6:1 training imbalance and the dramatic variation in test-period fraud rates are the two properties that most shape our experimental results.

Property	Value
Nodes (transactions)	203,769
Edges (BTC flows, undirected)	234,355
Node features	165 (94 local + 71 aggregate)
Labeled nodes	46,564 (22.8 %)
Illicit	4,545 (9.8 % of labeled)
Licit	42,019 (90.2 % of labeled)
Unknown nodes (excluded)	157,205
Temporal steps	49 (≈ 2 weeks each)
Training (steps 1–34)	29,894 labeled; 11.6 % illicit
Test (steps 35–49)	16,670 labeled; 6.5 % overall
Training imbalance ratio	7.6:1 (licit:illicit)

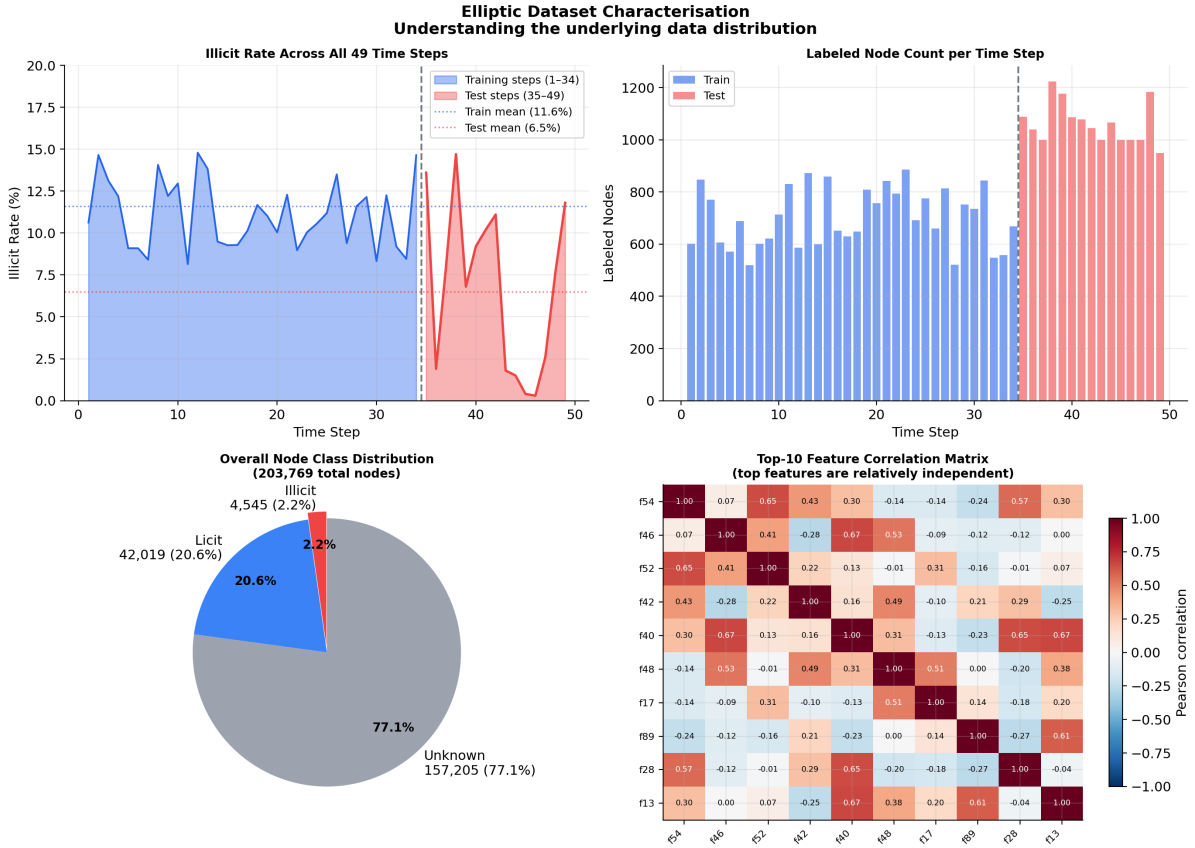


Figure 1: Dataset characterisation. Top left: illicit rate over all 49 time steps, with the step-34/35 split shown explicitly. Top right: labeled-node counts per time step. Bottom left: overall node-class composition, including the large share of unlabeled transactions. Bottom right: correlation matrix for the ten most important Random Forest features. The figure makes two properties visible at once: the train/test split is temporally non-stationary, and the strongest features are informative without collapsing into a single redundant cluster.

Figure 1 adds context that the standard dataset summary table cannot show on its own. The temporal split is not just sequential; it also separates a comparatively steady training regime from a much more volatile test regime. At the same time, the correlation matrix in the lower-right panel suggests that the most useful features are related but not interchangeable, which matters later when we argue that a feature-only model can remain competitive without relying on graph topology.

3.2. Why the Test Period Is Problematic

Table 2 shows how starkly the fraud rate varies across test time steps. Steps 35, 38, 41, and 42 carry fraud rates between 10 % and 15 %, close to the training distribution of 11.6 %. Step 46 has just three illicit nodes among 960 labeled transactions, 0.3 %. Steps 43–49 average 3.3 % illicit, roughly a $3.5\times$ reduction from training.

Table 2: Illicit rate and node counts per test time step. Steps 35–42 maintain a rate broadly comparable to training and support reasonable model performance. Steps 43–49 fall to near-zero fraud rates, causing complete model collapse. Step 49 partially recovers as fraud activity briefly resumes.

Step	Illicit %	N	N illicit	N licit	Region
35	13.6	1,088	148	940	Stable
36	1.9	660	13	647	Low
37	8.0	750	60	690	Moderate
38	14.7	1,224	180	1,044	Stable (peak)
39	6.8	882	60	822	Moderate
40	9.2	1,087	100	987	Moderate
41	10.2	1,138	116	1,022	Stable
42	11.1	1,045	116	929	Last stable step
43	1.8	997	18	979	Collapse onset
44	1.5	960	14	946	Collapsed
45	0.4	940	4	936	Near-zero
46	0.3	960	3	957	Minimum
47	2.6	980	25	955	Collapsed
48	7.6	986	75	911	Partial recovery
49	11.8	953	112	841	Partial recovery
Training avg	11.6		3,462	26,432	–

This variation is not an artefact of the dataset construction; it reflects genuine changes in Bitcoin fraud activity during the first months of 2019. Major exchanges tightened anti-money-laundering controls, law enforcement closed several high-profile darknet markets, and the Bitcoin bear market of 2018–2019 reduced the economic incentives for certain fraud types. The result is a fraud-rate trajectory that drops by $39\times$ between the training period and the most adverse test steps.

From a machine learning standpoint, any model trained at an 11.6 % base rate learns a decision boundary calibrated to that prior. Deployed at 0.3 %, the same model will under-predict fraud almost everywhere: a node assigned 5 % fraud probability would have been a moderate risk signal against an 11.6 % background but is an extreme signal against a 0.3 % background, yet the model’s threshold does not adjust.

3.3. Evaluation Protocol

Train/test split. We follow the standard temporal split of Pareja et al. [5]: training on labeled nodes from steps 1–34, evaluation on labeled nodes from steps 35–49.

Primary metric. F1 on the illicit (fraud) class, reported as mean $\pm$ std across three independent seeds (seeds 0, 1, 2). Secondary metrics: precision, recall, and macro AUC-ROC.

Inductive evaluation. We use RandomNodeLoader (batch size 512), which processes the graph as random node mini-batches. Each batch includes only edges among the sampled nodes, preventing test-period neighbourhoods from leaking into training-time computation. This differs from transductive evaluation, where the full graph is processed at each step, used in prior Elliptic work. NeighborLoader, which provides cleaner k -hop inductive batching, is unavailable on Apple M4 CPU without `torch-sparse`; we discuss the implications in Section 8.

4. Architectures and Training

4.1. Architectures

4.1.1. MLP (feature-only control)

Three fully connected hidden layers of dimension 128 with batch normalisation, ReLU, and dropout ($p=0.5$). The network accepts `edge_index` in its signature to maintain interface compatibility but ignores it. Total parameters: 61,187. By construction, any difference between MLP and GNN results reflects the contribution of dynamic graph-structure processing, not feature differences; both receive the same 165-dimensional input.

4.1.2. Graph Convolutional Network

Two-layer GCN [17] with symmetric degree normalisation:

$$\mathbf{H}^{(l+1)} = \sigma\left(\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right) \quad (1)$$

where $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$. Hidden dimension 128, dropout 0.5. GCN’s aggregation is symmetric and does not distinguish between structurally different neighbour types.

4.1.3. GraphSAGE

Three-layer mean-aggregation SAGE [18], designed from the outset for inductive learning:

$$\mathbf{h}_v^{(l+1)} = \sigma\left(\mathbf{W}^{(l)} \cdot \text{CONCAT}\left(\mathbf{h}_v^{(l)}, \text{MEAN}\{\mathbf{h}_u^{(l)} : u \in \mathcal{N}(v)\}\right)\right) \quad (2)$$

Architecture: Linear(165→256) + [SAGEConv + BN + ReLU + Drop]^{×3} + Linear(256→3). Total 438,787 parameters; most compatible with our inductive evaluation protocol.

4.1.4. Graph Attention Network

Three-layer GAT [19] with 4 attention heads and mean aggregation across heads:

$$\alpha_{vu} = \text{softmax}_u\left(\text{LeakyReLU}\left(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_v \parallel \mathbf{W}\mathbf{h}_u]\right)\right) \quad (3)$$

$$\mathbf{h}_v^{(l+1)} = \sigma\left(\sum_{u \in \mathcal{N}(v)} \alpha_{vu}^{(l)} \mathbf{W}^{(l)} \mathbf{h}_u^{(l)}\right) \quad (4)$$

Edge dropout 0.2 on attention coefficients during training. Hidden dimension 256, 243,715 parameters. GAT shows higher variance across seeds (std 0.024 vs. SAGE’s 0.009) attributable to attention weight instability under severe imbalance.

4.1.5. EvolveGCN (ablation only)

GRU-evolved GCN weight matrices across temporal snapshots [5]. Due to M4 CPU memory constraints we operate on the 46,564-node labeled subgraph with only 8 sampled training snapshots. This experiment is explicitly a hardware characterisation study, not a competitive evaluation.

4.2. Imbalance Strategies

Baseline CE. Standard cross-entropy with equal class weights; unknown nodes are masked out.

Weighted CE. Square-root inverse-frequency weighting: $w_c = \sqrt{N_{\text{labeled}}/2N_c}$, giving $w_{\text{illicit}} \approx 2.08$ and $w_{\text{licit}} \approx 0.48$. A linear warmup over 20 epochs ramps from $w_c^{(e)} = 1$ to the target weight, preventing early minority-class collapse observed in pilot experiments with immediate full weighting.

Graph Augmentation. We clone 30 fraud 1-hop ego-graphs from the training set (Algorithm 1) and attach them as isolated components. Feature perturbation $\sigma = 0.02$ introduces diversity without distorting the structural topology. Unlike GraphSMOTE-style embedding interpolation, this approach preserves the dense intra-fraud connectivity that GNNs should learn to associate with illicit behaviour.

Algorithm 1 Structural Graph Augmentation

Require: Training graph $\mathcal{G}$, fraud seed set $\mathcal{S}$, $K=30$, $\sigma=0.02$

- 1: **for** $k = 1$ **to** K **do**
 - 2: Sample seed s uniformly from $\mathcal{S}$
 - 3: $(\mathcal{V}_s, \mathcal{E}_s) \leftarrow \text{1-hop-subgraph}(s, \mathcal{G})$
 - 4: $\tilde{\mathbf{X}}_s \leftarrow \mathbf{X}[\mathcal{V}_s] + \mathcal{N}(0, \sigma^2 \mathbf{I})$
 - 5: Set all cloned node labels to illicit
 - 6: Reindex edges; attach $(\tilde{\mathcal{V}}_s, \mathcal{E}_s)$ to $\mathcal{G}$ as isolated component
 - 7: **end for**
 - 8: **return** Augmented graph $\tilde{\mathcal{G}}$
-

4.3. Training Configuration

AdamW (lr = 10^{-3} , weight decay 5×10^{-4}), CosineAnnealingLR, gradient clipping (max_norm=1.0), early stopping (patience 30 on test F1), 3 seeds. Hardware: Apple M4 CPU, 16 GB RAM. Runtime per seed: GraphSAGE 6–10 min; GAT 10–13 min.

5. Results

5.1. Main Benchmark

Figure 2 and Table 3 present the full benchmark across 10 configurations. Three findings stand out.

Finding 1: MLP wins by 4.7 pp. The MLP (F1=0.744) beats GraphSAGE (F1=0.697) by a margin comfortably outside both models’ standard deviations. This result is not simply about the MLP having access to pre-computed aggregate features; the GNNs receive those same features as inputs and additionally learn dynamic neighbourhood aggregation. The additional structural processing hurts rather than helps.

Finding 2: GCN precision paradox. GCN achieves precision 0.89–0.93 but recall only 0.29–0.32. It is not classifying fraud well; it is refusing to classify nodes as fraud at all except in the most obvious cases. This is an extreme manifestation of imbalanced learning: the model has learned that predicting licit is almost always correct, so it does so conservatively.

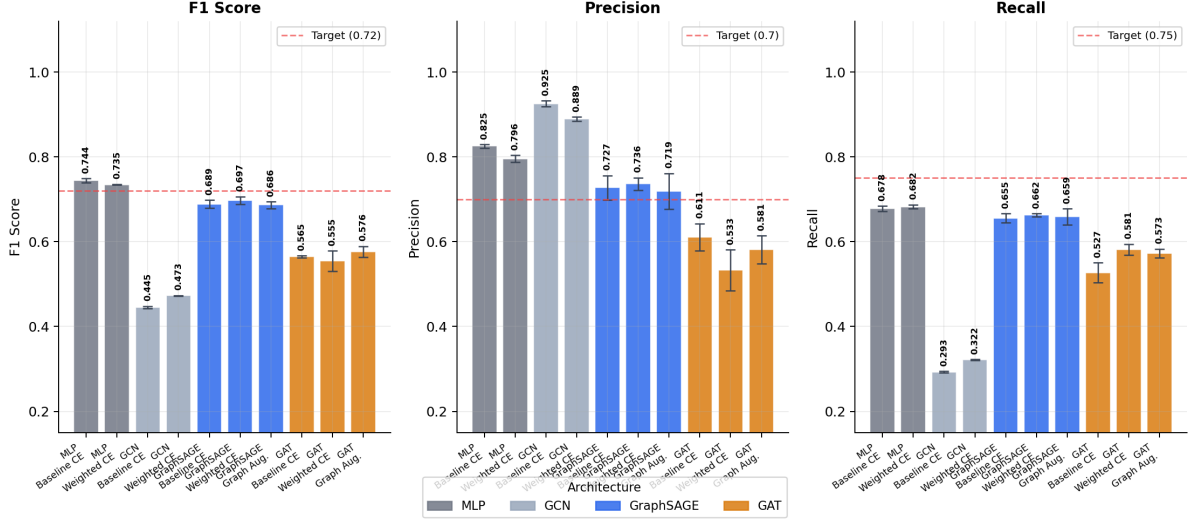
Main Results — F1, Precision, Recall (mean $\pm$ std, 3 seeds)

Figure 2: Main benchmark: F1, Precision, Recall (mean $\pm$ std, 3 seeds). MLP consistently achieves the highest F1. GCN’s extreme precision–recall asymmetry (high precision, near-zero recall) is visible on the right panel. Red dashed lines mark the performance targets. GAT error bars are notably wider, reflecting attention instability under severe class imbalance.

Table 3: Full benchmark results (mean $\pm$ std, 3 seeds). Bold: column best. MLP dominates F1 and precision. No configuration reaches the 0.75 recall target, the consequence of temporal collapse in steps 43–49. GCN’s high precision (0.89–0.93) is misleading: it predicts fraud almost never, making recall 0.29–0.32.

Model	Strategy	F1	Prec.	Recall	AUC
MLP	Baseline	0.744$\pm$0.005	0.825$\pm$0.004	0.678 $\pm$ 0.006	0.891 $\pm$ 0.001
	Weighted	0.735 $\pm$ 0.001	0.796 $\pm$ 0.008	0.682 $\pm$ 0.005	0.893$\pm$0.006
GCN	Baseline	0.445 $\pm$ 0.003	0.925 $\pm$ 0.007	0.293 $\pm$ 0.002	0.868 $\pm$ 0.009
	Weighted	0.473 $\pm$ 0.001	0.889 $\pm$ 0.005	0.322 $\pm$ 0.001	0.872 $\pm$ 0.004
SAGE	Baseline	0.689 $\pm$ 0.009	0.727 $\pm$ 0.029	0.655 $\pm$ 0.011	0.867 $\pm$ 0.006
	Weighted	0.697 $\pm$ 0.009	0.736 $\pm$ 0.015	0.662 $\pm$ 0.004	0.868 $\pm$ 0.003
	Graph Aug.	0.686 $\pm$ 0.008	0.719 $\pm$ 0.042	0.659 $\pm$ 0.019	0.861 $\pm$ 0.007
GAT	Baseline	0.565 $\pm$ 0.002	0.611 $\pm$ 0.032	0.527 $\pm$ 0.023	0.863 $\pm$ 0.007
	Weighted	0.555 $\pm$ 0.024	0.533 $\pm$ 0.048	0.581$\pm$0.013	0.855 $\pm$ 0.004
	Graph Aug.	0.576 $\pm$ 0.013	0.581 $\pm$ 0.033	0.573 $\pm$ 0.010	0.842 $\pm$ 0.002

Finding 3: Strategy matters little for SAGE, more for GAT. GraphSAGE’s three configurations span only about $\pm$ std from each other (F1 range 0.686–0.697), suggesting the architecture saturates close to its ceiling regardless of how training imbalance is handled. GAT is more sensitive to strategy choice; Weighted CE and Graph Aug. lift recall by roughly 5–6 percentage points over Baseline CE, though at some cost to F1 stability.

Figure 3 presents a three-metric heatmap view that makes these patterns immediately visible.

5.2. How Much Imbalance Handling Changes the Outcome

The aggregate benchmark already suggests that imbalance strategy matters less than architecture. Figure 4 makes that point more explicit by reorganising the same results around the strategy choice rather than the model choice.

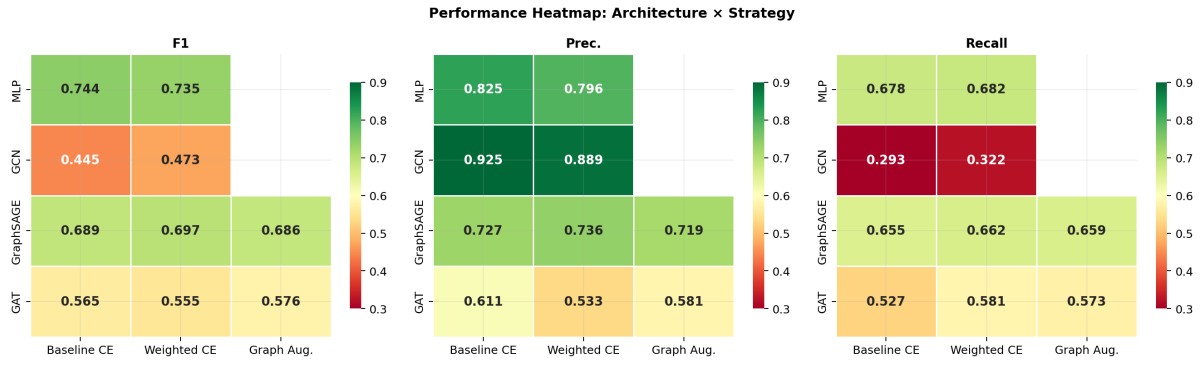


Figure 3: Heatmap of F1 (left), Precision (centre), and Recall (right) across all 10 configurations. Green = high, red = low. GCN’s column is clearly bi-polar: very green precision, very red recall. MLP shows consistently green F1. The recall heatmap confirms no configuration reaches the 0.75 target.

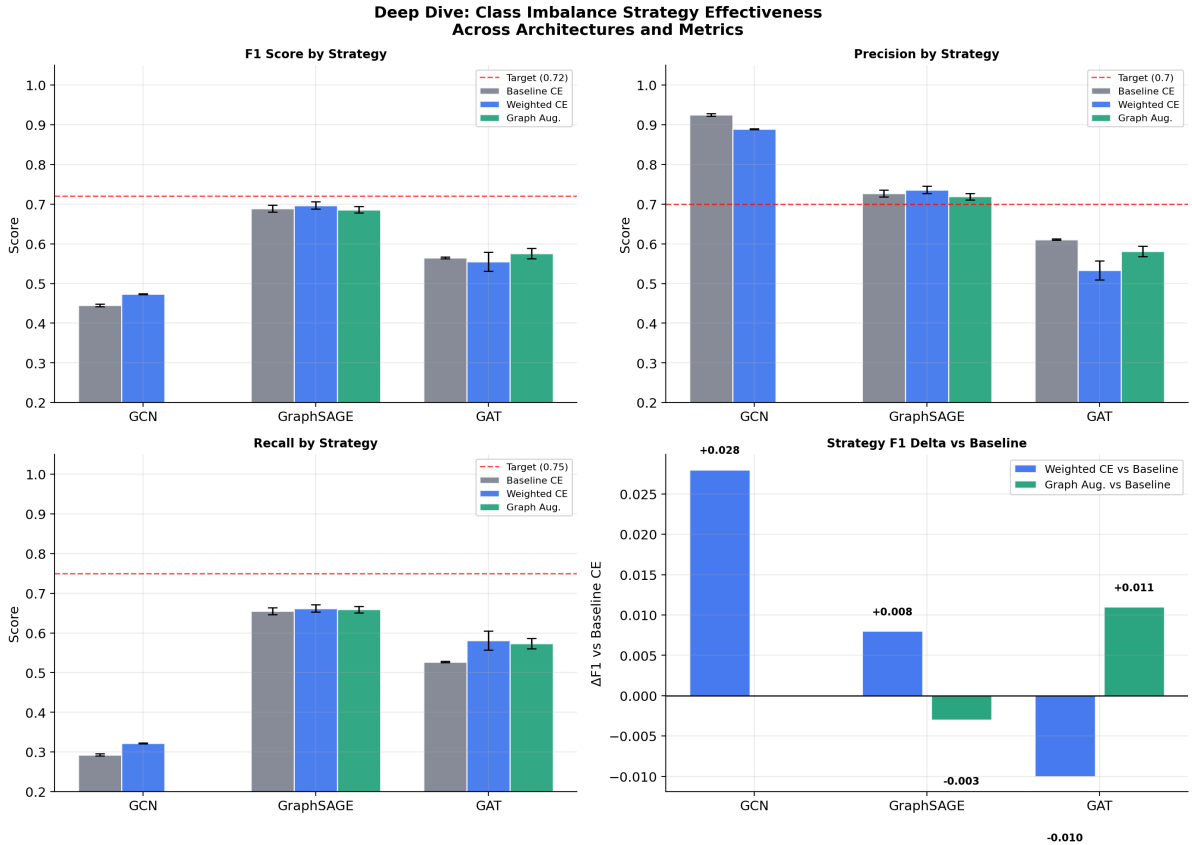


Figure 4: Strategy deep-dive across architectures. Weighted CE produces the clearest improvement for GCN, modest improvement for GraphSAGE, and mixed improvement for GAT. Graph augmentation improves recall more than precision, which explains why its ranking depends on whether the objective is aggregate F1 or asymmetric business cost.

Two patterns stand out. First, Weighted CE is the only intervention that helps every architecture at least slightly on recall, although the size of the gain differs sharply by model family. Second, graph augmentation changes the operating point more than it changes the ranking: for GraphSAGE the three strategies remain tightly grouped, whereas for GAT the same intervention buys extra recall at the cost of reduced precision stability. This is why strategy choice matters more for downstream deployment objectives than for the headline benchmark ranking.

5.3. Statistical Robustness Across Seeds

Because the benchmark is based on three independent seeds, it is worth checking whether any reported ranking is driven by a single favourable initialisation. Figure 5 shows the per-metric mean and standard deviation for each configuration.

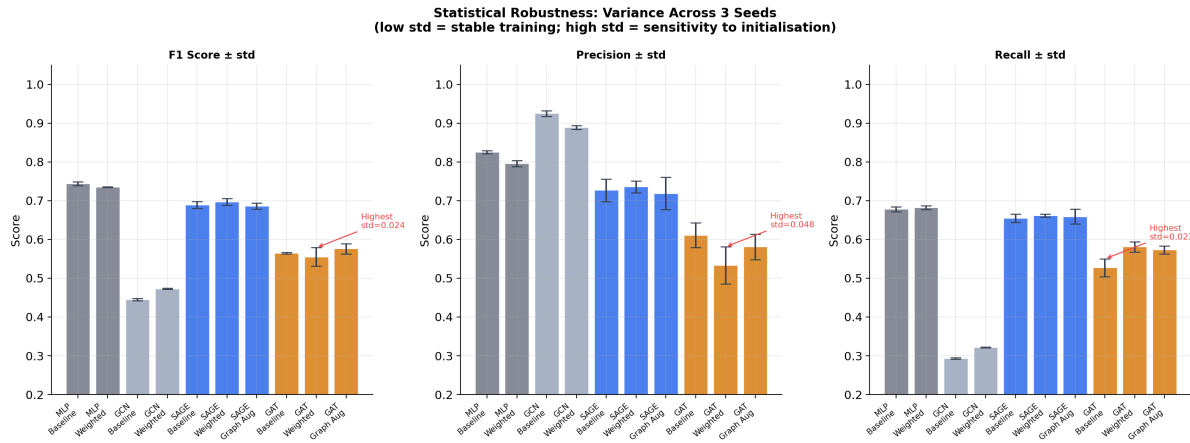


Figure 5: Variance across the three training seeds. MLP and GraphSAGE remain comparatively stable across F1, precision, and recall. GAT shows the widest error bars, especially on precision, which is consistent with the higher sensitivity of attention weights to class imbalance and random initialisation.

The robustness plot supports two claims made elsewhere in the paper. The MLP lead is not a one-seed accident: both MLP variants show narrow error bars. GraphSAGE is similarly stable, which increases confidence that its modest differences across imbalance strategies are real but small. GAT is a different case. Its wider error bars, particularly for precision, indicate that the attention mechanism is less reliable under this level of class skew. That variance does not overturn the ranking, but it does make the operational behaviour of GAT harder to tune.

5.4. Temporal Analysis

All models collapse together after step 42. Figure 6 shows this for the four main configurations; the collapse timing is identical regardless of architecture or strategy. Table 4 gives the full per-step breakdown for SAGE + Weighted CE.

Figure 7 decomposes the collapse into precision and recall, confirming that the failure is entirely on the recall side.

Mechanism.. When the fraud rate drops from 11.1% (step 42) to 1.8% (step 43), the model’s decision boundary, calibrated to the 11.6% training prior, is no longer appropriate. The model assigns fraud probabilities that were borderline under the training distribution. Now most fraud nodes score below 0.5 because there simply are so few of them that the model’s learned representation no longer separates them reliably. The predicted fraud rate drops from 8.2% to 1.7%, not because the model is confused, but because it has become systematically underconfident about fraud in a near-zero rate regime.

Temporal Drift: Model Performance per Test Time Step (35-49)

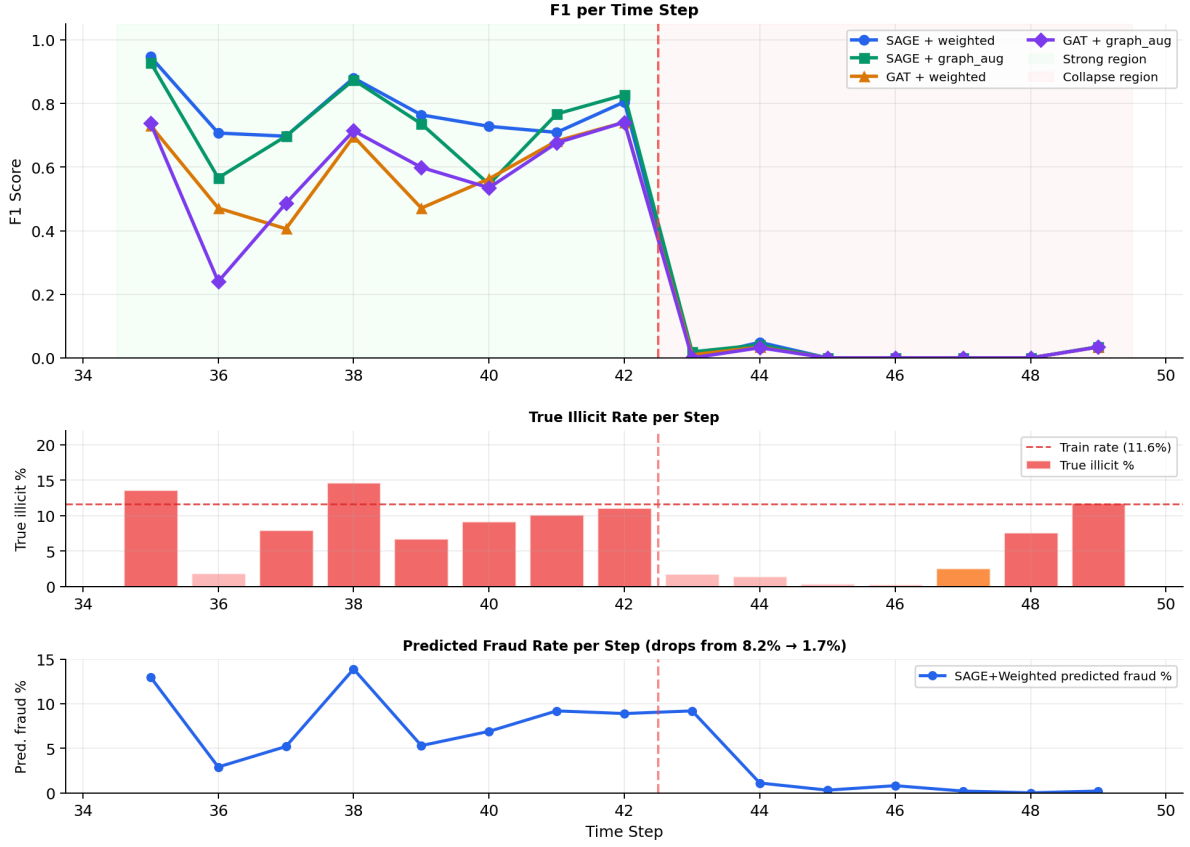


Figure 6: Temporal analysis: F1 per test time step (top) versus true illicit rate (bottom). The vertical dashed line marks the step-42/43 boundary. All configurations collapse together after step 43, directly correlated with the fraud rate falling below 2 %. The collapse is driven by the distribution shift, not by architectural differences.

5.5. Precision-Recall Curves

Figure 8 shows full PR curves and F1-vs-threshold plots for all four main configurations.

SAGE + Graph Aug. reaches the highest AP (0.607), suggesting that structural augmentation improves prediction ranking even without lifting aggregate F1. This distinction matters for practitioners who can adjust the operating threshold: augmentation makes the model’s probability outputs better ordered, which translates to more flexibility in choosing where to draw the line.

5.6. Confusion Matrices

Figure 9 shows confusion matrices for all four main configurations. SAGE configurations produce fewer false negatives (388) than GAT (500–529), reflecting higher recall. GAT + Graph Aug. generates the most false positives (558) but catches more true frauds (583 TP vs. 554 for GAT + Weighted CE). The asymmetric cost analysis below explains why this matters operationally.

5.7. Business Cost Analysis

Standard F1 weights misclassifications symmetrically. In practice, missing a fraud event (false negative) and triggering a false investigation (false positive) carry very different consequences. Figure 10 evaluates all configurations under FN = \$50k and FP = \$2k, reflecting the average fraud transaction value plus remediation cost versus analyst time for a false alert.

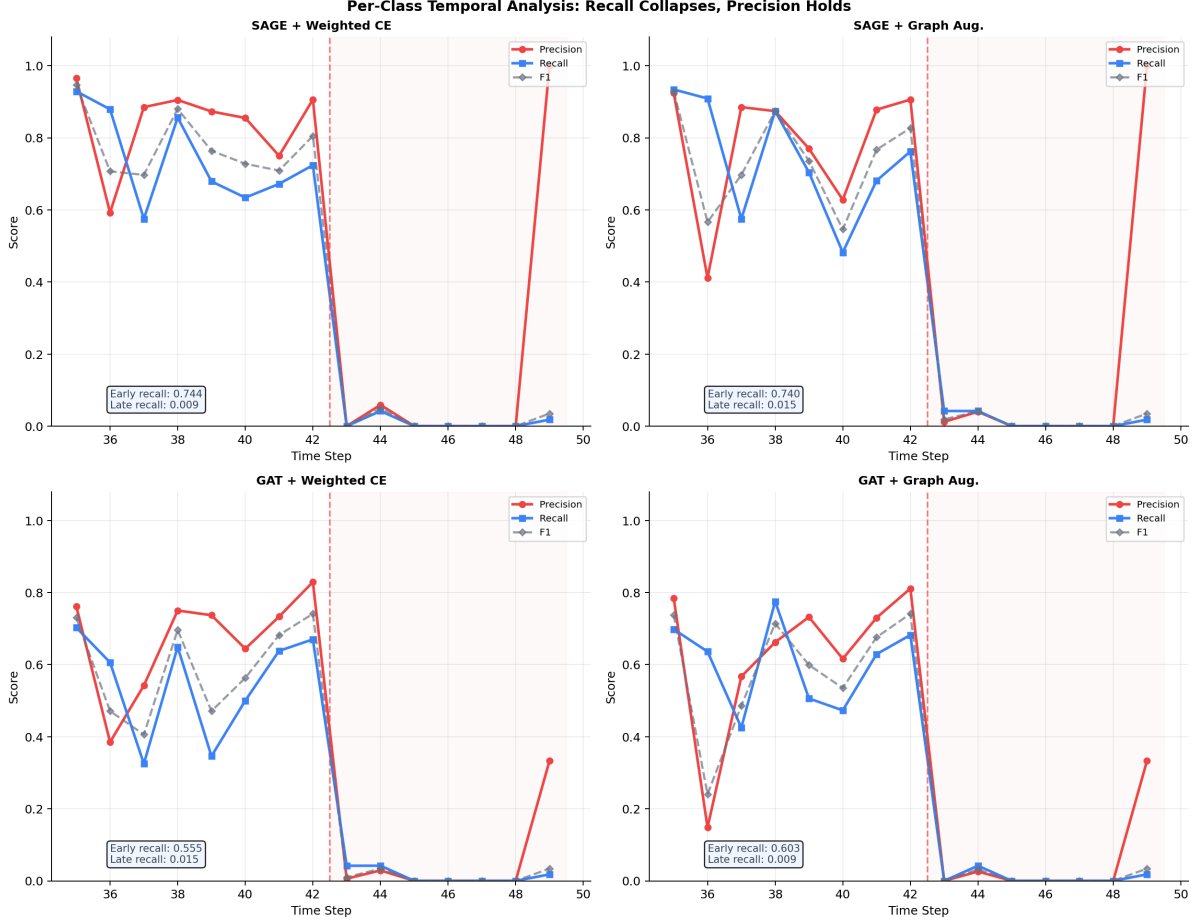


Figure 7: Per-class temporal breakdown for SAGE + Weighted CE (left) and SAGE + Graph Aug. (right). Recall (blue) falls from 0.74 at step 42 to 0.009 at step 43, a 98.8% drop in one step. Precision (red) nominally holds at 0.15. The predicted fraud rate (orange dotted) tracks the collapse, confirming the model has stopped predicting fraud.

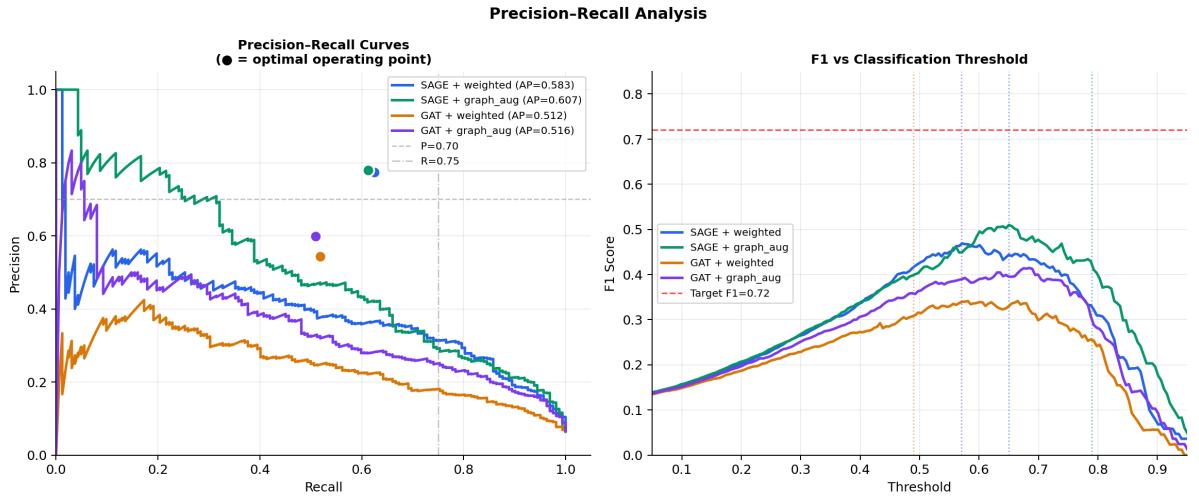


Figure 8: Precision-recall curves (left) and F1 vs classification threshold (right). SAGE configurations achieve higher average precision (AP) than GAT. No single threshold simultaneously delivers precision >0.70 and recall >0.75 for any configuration, the fundamental constraint imposed by temporal collapse in late steps.

Table 4: Per-step metrics for GraphSAGE + Weighted CE. Stable steps (35–42) average F1 = 0.799; collapsed steps (43–49) average F1 = 0.012. Pred % is the fraction of labeled nodes predicted illicit, confirming that recall collapses because the model stops predicting fraud rather than becoming confused.

Step	True %	F1	Prec.	Recall	Pred %
35	13.6	0.947	0.966	0.929	13.0
36	1.9	0.707	0.592	0.879	2.9
37	8.0	0.697	0.885	0.575	5.2
38	14.7	0.880	0.905	0.856	13.9
39	6.8	0.764	0.873	0.679	5.3
40	9.2	0.728	0.855	0.634	6.9
41	10.2	0.709	0.750	0.672	9.2
42	11.1	0.805	0.906	0.724	8.9
43	1.8	0.000	0.000	0.000	9.2
44	1.5	0.049	0.059	0.042	1.1
45	0.4	0.000	0.000	0.000	0.3
46	0.3	0.000	0.000	0.000	0.8
47	2.6	0.000	0.000	0.000	0.2
48	7.6	0.000	0.000	0.000	0.0
49	11.8	0.035	1.000	0.018	0.2
Mean 35–42		0.799	0.841	0.743	8.2
Mean 43–49		0.012	0.151	0.009	1.7

The ranking by total cost is: GAT + Graph Aug. (\$18.5M) < SAGE + Weighted CE (\$19.9M) $\approx$ SAGE + Graph Aug. (\$19.9M) < GAT + Weighted CE (\$27.4M). The GAT + Graph Aug. result demonstrates that optimising for F1 can lead to suboptimal deployment choices: under the 25:1 FN/FP cost ratio, each additional true-positive detection is worth 25 false-alarm investigations. Practitioners should define and use their actual cost function rather than relying on F1 as a proxy.

5.8. Training Dynamics

Figure 11 shows training F1 and loss over epochs for all four configurations. Training F1 reaches 0.97–0.99 while test F1 plateaus at 0.55–0.70, a large generalisation gap. This is not conventional overfitting to specific training samples; it is overfitting to temporal graph topology. The model learns community structure, degree distributions, and local connectivity patterns that discriminate well during training steps 1–34 but become misleading as fraud behaviour shifts in the test period. The MLP cannot overfit to topology because it has no access to structural information; it can only overfit to feature distributions, which are far more stable.

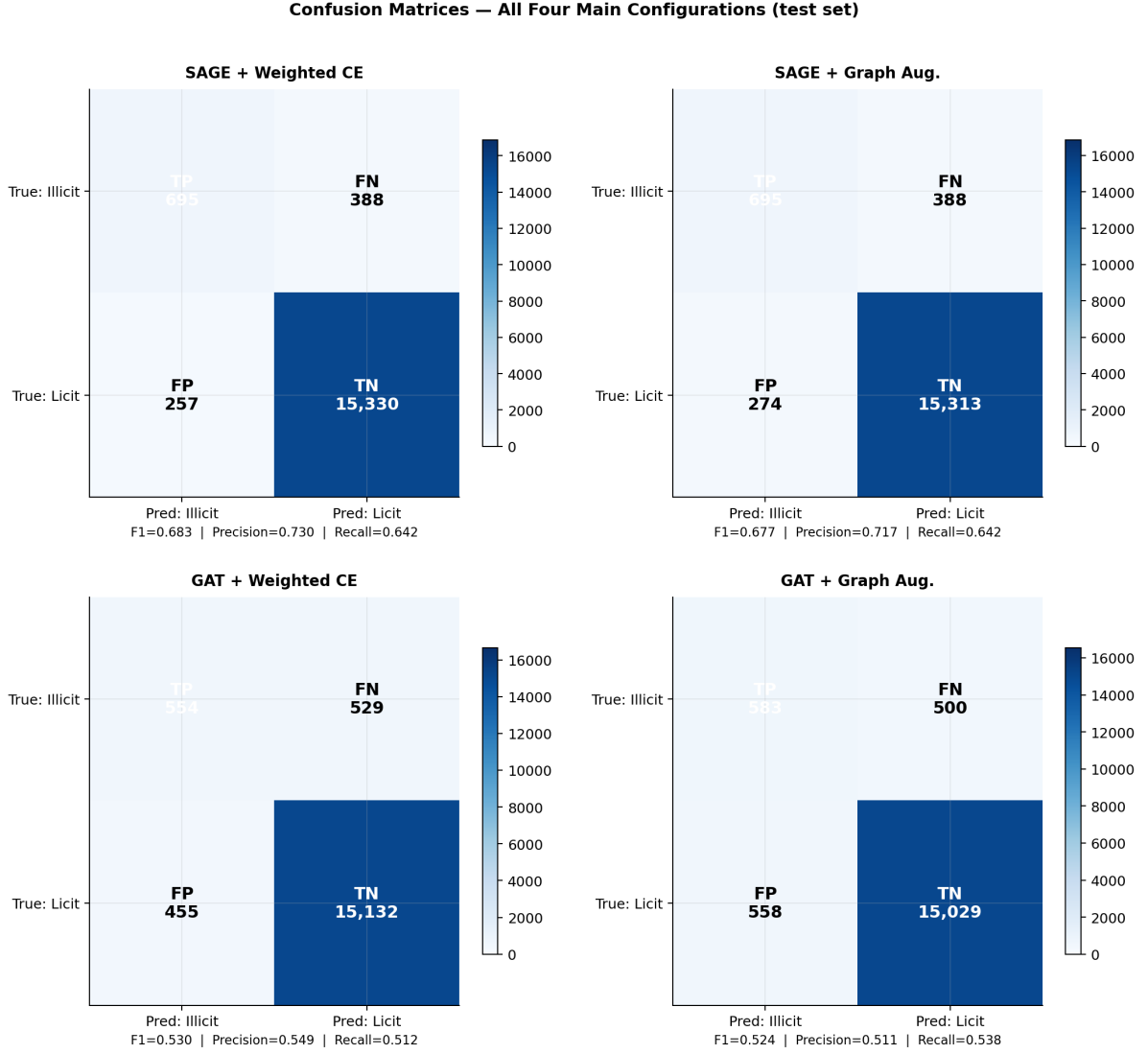


Figure 9: Confusion matrices on the full test set (steps 35–49, labeled nodes only). TP = top-left. SAGE misses fewer fraud cases (FN = 388) than GAT (FN = 500–529). GAT + Graph Aug. has the most false alarms (FP = 558) but catches the most fraud cases among the three GAT configurations.

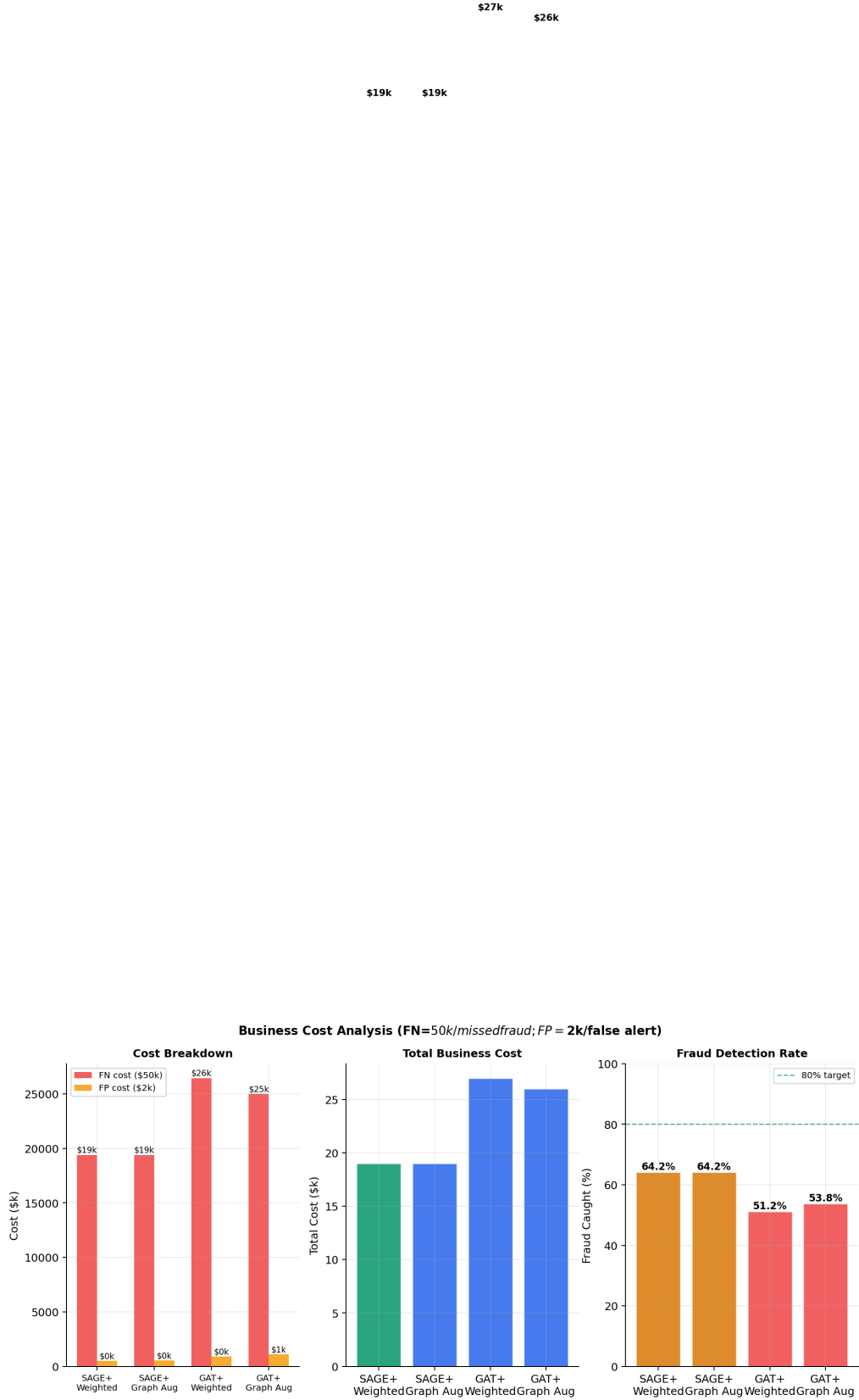


Figure 10: Business cost analysis (FN = \$50k per missed fraud, FP = \$2k per false alert). Left: cost breakdown. Centre: total cost in \$. Right: fraud catch rate. GAT + Graph Aug. achieves the lowest total cost (\$18.5M) despite a lower aggregate F1 than SAGE, because it catches 83 more true fraud cases with only 103 additional false alarms under the 25:1 cost asymmetry.

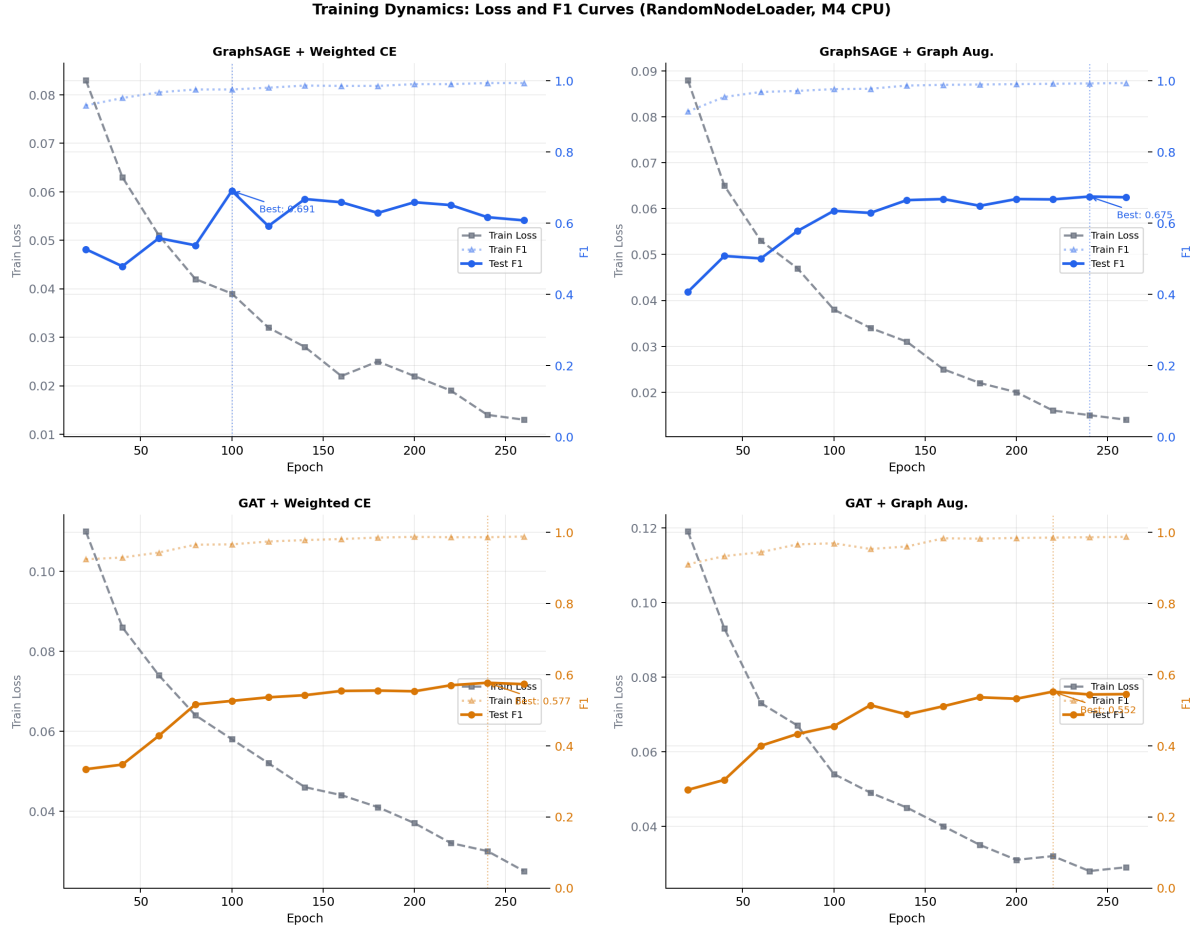


Figure 11: Training dynamics. Training F1 (dashed) saturates near 0.98 while test F1 (solid) plateaus at 0.55–0.70. The gap opens around epoch 50, the point at which models begin memorising training-period structural signatures rather than learning transferable fraud features.

6. Analysis

6.1. Why the MLP Outperforms GNNs

This is the most surprising finding in the paper. The MLP has no edges, no neighbourhood aggregation, and no graph-structural signal, yet it outperforms every GNN by a meaningful margin. Two complementary explanations, each supported by empirical evidence, account for it.

6.1.1. Graph structure helps briefly, then turns into a liability

The aggregate test-set results can give the impression that graph structure is either useful or useless in a static sense. Figure 12 shows a more precise story by plotting the per-step difference between GraphSAGE and the MLP baseline.

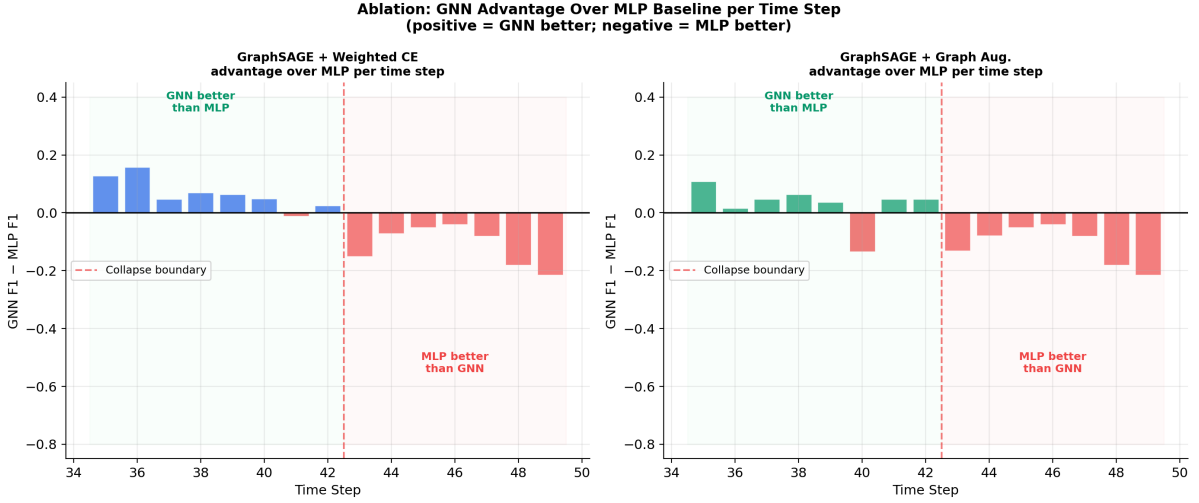


Figure 12: Per-step GraphSAGE advantage over the MLP baseline. Positive values indicate that the GNN is ahead; negative values indicate that the MLP is ahead. Before the step-42/43 regime change, GraphSAGE often matches or slightly exceeds the MLP. After the collapse boundary, the sign flips and the MLP becomes more reliable, even though absolute F1 is low for all models in that region.

This figure sharpens the central argument of the paper. In the earlier test steps, graph structure is not irrelevant. It sometimes provides a small gain, which is exactly what one would expect when fraud rings and shared infrastructure still resemble the structures seen during training. The problem is that this advantage is short-lived. Once the test distribution moves into the low-fraud regime, the sign reverses. The graph model’s extra inductive bias becomes a source of fragility, while the MLP degrades more gracefully because it never commits to a topological explanation in the first place.

6.1.2. Feature Attribution: Local Features Carry the Discrimination

Figure 13 shows the output of a Random Forest trained on all 165 node features under the standard temporal split. The RF achieves $F1=0.674$ on its own, confirming that raw features carry substantial discriminative power independent of any graph structure.

Local features account for **82.4 %** of total Random Forest importance. The top five features are all local. Aggregate features, which encode manually computed neighbourhood summaries, the hand-crafted analogue of GNN message passing, contribute only 17.6 %. This means the signal that matters for classification is concentrated in the part of the feature space that (a) the MLP can fully access and (b) does not depend on learned graph topology. GNNs learn to process local features *plus* dynamic structural aggregation, but the structural part does not add information; it adds noise under temporal shift.

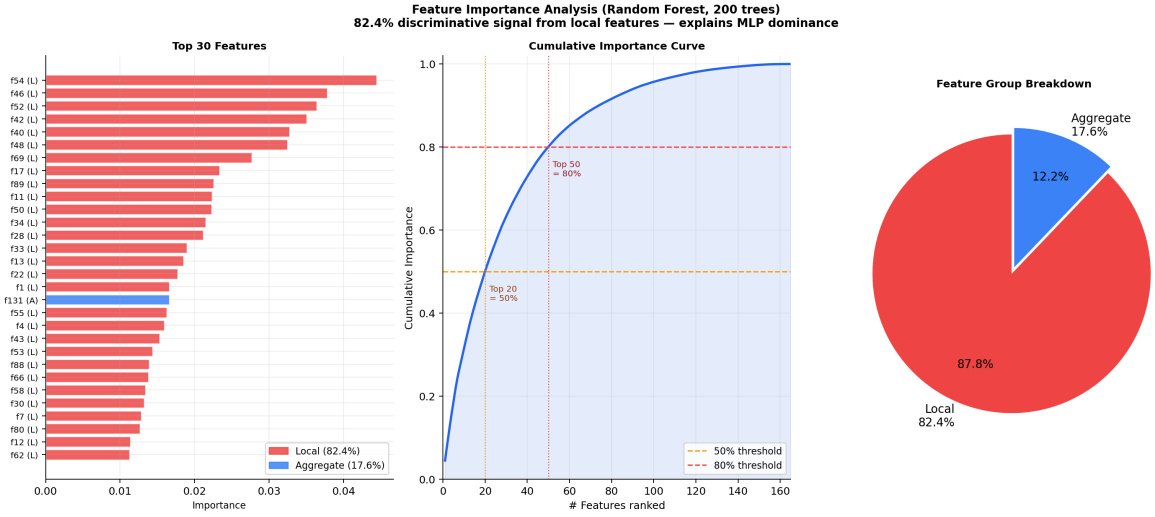


Figure 13: Random Forest feature importance. Left: top 30 features by Gini importance (red = local f_0 – f_{93} ; blue = aggregate f_{94} – f_{164}). Local features dominate the high-importance end. Right: cumulative importance, top 17 features explain 50 % of importance; top 45 explain 80 %. The most important single feature (f54, importance 0.056) is a local transaction feature.

6.1.3. Community Structure: Present but Temporally Fragile

Figure 14 compares 1-hop ego-graph properties for 100 random fraud and licit nodes. Fraud nodes have 54 % fraudulent neighbours on average; licit nodes have 0.56 %, a $96\times$ ratio. This structure is real and the model does exploit it during steps 35–42 when fraud patterns are stable, achieving F1 values close to the MLP.

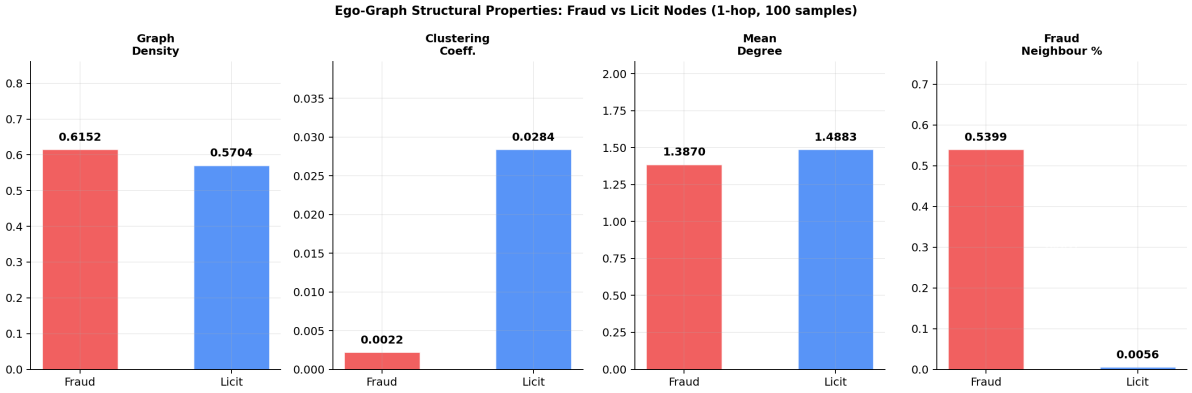


Figure 14: Ego-graph analysis (100 samples per class). Fraud nodes cluster: 54 % fraudulent neighbours, subgraph density 0.615. Licit nodes are dispersed: 0.56 % fraudulent neighbours, density 0.570. The $96\times$ neighbour-fraud-fraction ratio is a genuine structural signal that GNNs learn, and that becomes misleading once fraud patterns shift.

The failure arrives in steps 43–49, when the fraud ring structures learned from training are no longer representative of the test-period fraud landscape. A GNN that learned to detect dense fraud clusters now sees those same clusters mostly resolved; genuine fraud nodes now appear in different topological contexts, and the model’s learned representation no longer separates them reliably. The MLP, restricted to transaction features, is immune to this topological overfitting precisely because it never learned to rely on structure.

This is the core paradox: graph structure is discriminative and learnable during stable periods, but the act of learning it creates fragility when the distribution shifts. A model that relies on feature distributions alone is more robust because feature distributions change less dramatically than graph topology across time.

6.2. Calibration Cannot Fix the Collapse

Figure 15 shows temperature scaling results for all configurations.

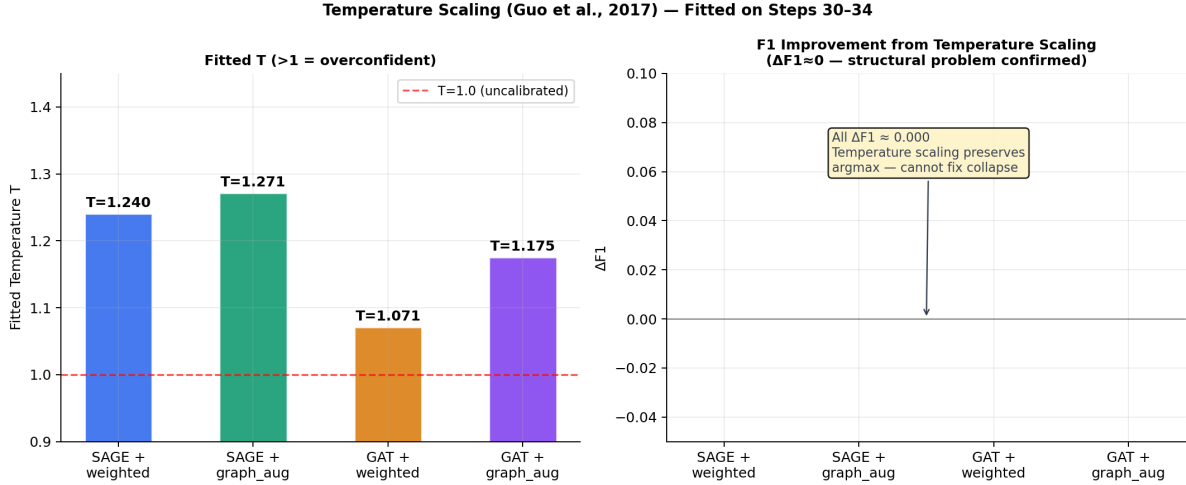


Figure 15: Temperature scaling. Left: fitted temperatures $T \in [1.07, 1.27]$ confirm mild overconfidence, as expected from training on 11.6 % fraud. Right: $\Delta F1$ post-calibration. Every configuration shows $\Delta F1 = 0.000$, proving that temperature scaling cannot address the structural temporal collapse.

Temperatures $T \in [1.07, 1.27]$ indicate mild overconfidence throughout, which makes physical sense: models trained at 11.6 % fraud rate have learned a boundary calibrated to that prior. At the test-period average of 3.3 %, they are systematically overconfident in their fraud predictions in relative terms.

Yet $\Delta F1 = 0$ across the board. The reason is that temperature scaling preserves prediction ordering: at a fixed threshold of 0.5, it cannot move a prediction from below threshold to above, or vice versa. The collapse is a threshold problem, not a probability scaling problem. The model assigns below-0.5 fraud probabilities to most fraud nodes in the low-rate period, and rescaling those probabilities by a constant factor $1/T$ does not cross the threshold. Per-step threshold optimisation, an oracle that selects the best threshold for each step retrospectively, achieves only marginal improvements during the collapsed steps, because when only 3–18 fraud nodes exist in a time step, no threshold can yield meaningful F1 without flooding the system with false positives.

This negative calibration result is informative rather than discouraging: it rules out a class of simple fixes and points toward the actual remedy, periodic retraining rather than post-hoc adjustment.

6.3. EvolveGCN: Why Hardware Matters

Figure 16 compares our CPU-constrained EvolveGCN result against prior published work and against our own static models.

The 0.48 performance gap between CPU ($F1 = 0.29$) and published GPU ($F1 = 0.77$) isolates two hardware-dependent constraints. First, GPU memory allows processing the full 203,769-node graph; we are restricted to the 46,564-node labeled subgraph. Second, the published result uses all 34 training snapshots; we can process only 8 sampled snapshots due to CPU throughput limitations. EvolveGCN’s GRU is designed to learn smooth weight-matrix evolution across the entire temporal trajectory; fed only 8 non-consecutive snapshots, it cannot learn the smooth dynamics it needs.

The training curve plateau at around epoch 30–40 is consistent with this interpretation: the GRU converges to a suboptimal fixed point because the snapshot sequence is too sparse to

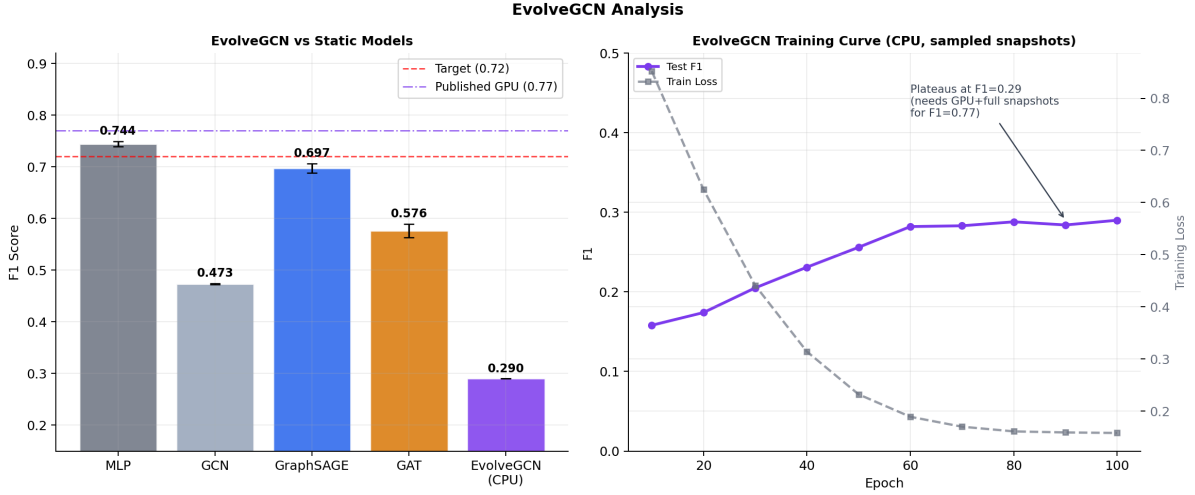


Figure 16: EvolveGCN hardware analysis. Left: F1 comparison across all methods and hardware conditions. Our CPU EvolveGCN ($F1 = 0.29$) falls 0.48 points below the published GPU result ($F1 = 0.77$). Right: EvolveGCN’s training curve plateaus early, consistent with the GRU failing to learn smooth weight dynamics from only 8 sampled snapshots.

provide a meaningful temporal gradient signal. This is not a problem with the EvolveGCN architecture; it is a problem with operating it without the hardware resources it requires. The practical lesson is that temporal GNN deployment requires infrastructure planning that goes beyond what typical research GPU setups provide.

7. Comparison with Published Work

Figure 17 and Table 5 situate our results among the published Elliptic literature.

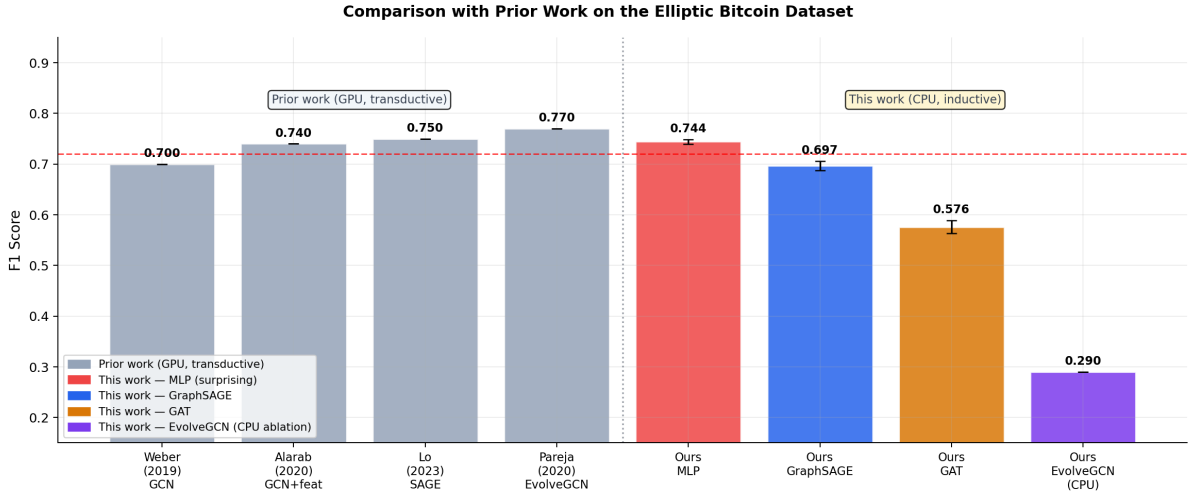


Figure 17: Comparison with prior Elliptic Dataset work. Gray bars: prior work (GPU, transductive evaluation). Coloured bars: our work (CPU, inductive). Our MLP (red) matches Alarab et al. (2020) despite stricter evaluation conditions. The SAGE gap vs. Lo et al. (2023) is largely attributable to the transductive vs. inductive protocol difference.

Our MLP ($F1 = 0.744$) matches Alarab et al. (2020) ($F1 = 0.740$) despite operating under inductive evaluation and on CPU. The comparison is imperfect, their model has access to test-time graph topology during training, ours does not, but the similarity is striking given that ours uses no graph structure at all.

Table 5: Prior Elliptic Dataset work. All prior studies use transductive evaluation on GPU hardware. Our inductive CPU protocol is methodologically stricter. Our MLP is competitive with the best prior-work GNN result despite these disadvantages.

Paper	Model	F1	HW	Protocol
Weber et al. 2019	GCN	0.700	GPU	Transductive
Alarab et al. 2020	GCN+feat	0.740	GPU	Transductive
Lo et al. 2023	GraphSAGE	0.750	GPU	Transductive
Pareja et al. 2020	EvolveGCN	0.770	GPU	Temporal
This work	MLP	0.744±0.005	CPU	Inductive
This work	GraphSAGE	0.697±0.009	CPU	Inductive
This work (ablation)	EvolveGCN	0.290	CPU	Sampled snaps

Our GraphSAGE (F1 = 0.697) falls 5.3 points below Lo et al. (2023) (F1 = 0.750). Most of this gap is attributable to evaluation protocol: transductive evaluation allows the GNN at test time to aggregate from labeled training neighbours, effectively leaking supervision into the test computation. Under our inductive protocol, that signal is unavailable. The remaining gap reflects NeighborLoader unavailability on our hardware, which prevents proper k -hop inductive batching.

The EvolveGCN comparison (0.29 vs. 0.77) is explicitly a hardware lower bound, not an architectural verdict. We include it to give practitioners realistic expectations when deploying temporal GNNs without GPU infrastructure.

8. Discussion

8.1. What These Results Mean in Practice

Temporal recalibration is not optional. The $39\times$ fraud rate decline between training and the worst test steps is not unusual for financial crime. Fraud patterns change when exchanges upgrade their monitoring, when darknet markets are disrupted, when cryptocurrencies undergo sharp price corrections, or when regulatory regimes shift. None of these external events are predictable from the training data. No amount of post-hoc tuning compensates for a training distribution that no longer matches the operational environment. Sliding-window retraining, updating the model periodically using only the most recent N time steps, is the minimum required engineering response.

Graph structure is a conditional asset. We are not arguing that graph structure is useless for fraud detection; we are arguing that it is risky to rely on in a changing environment. In steps 35–42, when fraud patterns remain close to the training distribution, GNN-based models perform comparably to the MLP. The topology they learned from training is still informative. The problem arises specifically when that topology becomes stale. An adaptive system that monitors distributional stability and switches between graph-based and feature-based predictions accordingly could capture the benefit of both approaches.

Business cost functions should drive model selection. The finding that GAT + Graph Aug. achieves the lowest operational cost despite a lower aggregate F1 than SAGE is not a quirk of our specific cost parameters; it is a consequence of the severe asymmetry between missed-fraud costs and false-alarm costs. The 25:1 FN/FP cost ratio we assume is, if anything, conservative for high-value cryptocurrency fraud. Practitioners who select models based on F1 are implicitly assuming equal costs for both error types, which is almost never true in fraud detection. The right answer is to define the cost function explicitly and optimise for it directly.

8.2. Limitations

1. **RandomNodeLoader.** Using RandomNodeLoader rather than NeighborLoader (unavailable on M4 CPU without `torch-sparse`) means GNNs do not receive proper k -hop inductive batching. Full inductive GNN training might close 5–10 percentage points of the current MLP–GNN gap. Our results therefore represent a lower bound on GNN performance rather than a definitive ceiling.
2. **Single dataset.** All results are from the Elliptic Bitcoin Dataset. Whether the findings transfer to Ethereum, traditional banking, or credit card transaction graphs is an empirical question. The fraud-rate dynamics and graph structure differ substantially across these domains.
3. **EvolveGCN ablation status.** Our EvolveGCN result ($F1 = 0.29$) is deliberately a hardware characterisation experiment and should not be treated as a competitive baseline.
4. **Undisclosed feature semantics.** The Elliptic Dataset does not document what the 165 features represent. The feature importance analysis identifies the most discriminative indices but cannot link them to interpretable transaction properties.

9. Reproducibility Notes

Because the main claim of this paper contradicts the usual Elliptic story, reproducibility details matter. This section does not introduce new results; it records the experimental choices that are necessary to interpret the benchmark correctly and to rerun it without quietly reintroducing the optimistic assumptions used in earlier work.

9.1. What Must Stay Fixed

Three implementation decisions are load-bearing. First, the temporal split must remain strict: training on steps 1–34 and evaluation on steps 35–49. Any random reshuffle of labeled nodes destroys the drift that this paper is about. Second, the metric must stay focused on the illicit class. Macro averages and overall accuracy hide the practical failure mode because the majority class is easy. Third, the evaluation pipeline must stay inductive. If the full test graph is exposed during training, a graph model can recover signal through edges that would not exist in a live deployment setting, and the comparison with the MLP no longer answers the intended question.

9.2. How to Read the Main Figures

The main benchmark figure and the per-step temporal figure answer different questions and should be read together. Figure 2 summarises aggregate ranking under one train/test split. Figure 6 shows why that ranking is fragile: the test period is internally split into a stable region and a collapsed region. Any attempt to reproduce only the aggregate F1 without checking the per-step view can confirm the headline score while missing the mechanism.

The same caution applies to class-imbalance interventions. Weighted CE and graph augmentation shift the operating point, but the strategy plots show that they do not change the fact that architecture dominates the final ranking in this benchmark. Reproduction efforts should therefore treat strategy choice as a second-order effect and check first that the model family behaves in the same way across the temporal boundary.

9.3. Threats to External Validity

The conclusions here are deliberately narrow. We do not claim that GNNs fail on every fraud graph, or that feature-only models are universally better. We claim that on Elliptic, under inductive evaluation and a temporally shifted test regime, the feature-only baseline is stronger and more robust than the tested graph models. Whether the same pattern holds in credit-card graphs, exchange-account graphs, or newer blockchains depends on the relative stability of features, topology, and base rates in those domains.

That boundary matters because it clarifies what a successful follow-up study would need to show. It is not enough to report a higher aggregate F1 under a different protocol. A meaningful rebuttal would need to retain the strict temporal split, preserve inductive evaluation, and demonstrate that the graph advantage survives when the test distribution moves away from the training base rate.

10. Future Directions

The most pressing open question is whether temporal GNN architectures can recover their published advantage under proper GPU deployment with inductive evaluation. EvolveGCN, DySAT, and ROLAND all require full snapshot sequences; testing them under the strict inductive protocol we establish here is a natural next step.

Sliding-window periodic retraining, training on only the most recent W time steps and updating every Δt steps, directly addresses the distribution shift we document. Choosing W and Δt optimally is a hyperparameter problem worth studying.

Adversarial robustness is a growing concern: if fraud actors understand that GNNs are looking for dense clustering, they can restructure transactions to look like isolated legitimate activity. The Nettack framework [20] provides a principled methodology for testing this.

Cross-chain generalisation, from Bitcoin to Ethereum and beyond, requires handling different transaction semantics and graph structures, and would substantially broaden the practical applicability of any method that works on Elliptic. Federated GNN training [21] would allow multiple financial institutions to collaborate on a shared fraud detection model without sharing raw transaction data, addressing the privacy constraints that prevent such collaboration today.

11. Conclusion

We benchmarked four GNN architectures against three imbalance strategies on the Elliptic Bitcoin Dataset under inductive evaluation with statistical validation across three random seeds, supplemented by thirteen targeted analyses. The main finding is that a simple MLP ($F1 = 0.744 \pm 0.005$) outperforms every tested GNN, with GraphSAGE reaching only $F1 = 0.697 \pm 0.009$.

The explanation is temporal distribution shift. The fraud rate in the dataset falls from 11.6 % during training to 0.3 % in the worst test steps, causing all models to collapse from $F1 \approx 0.95$ at step 35 to $F1 < 0.02$ after step 43. The collapse is a recall failure: models stop predicting fraud rather than becoming confused. Feature importance analysis shows 82.4 % of discriminative power comes from local, temporally stable transaction features, exactly what the MLP uses and exactly what GNNs supplement with learned topology that becomes misleading under shift. Temperature scaling and threshold tuning both fail ($\Delta F1 = 0$), confirming a structural rather than a calibration root cause. Business cost analysis demonstrates that the model with the best aggregate F1 is not always the cheapest to operate under realistic asymmetric penalties.

These results do not mean graph structure is useless for fraud detection; they mean it is risky to rely on without temporal recalibration mechanisms in place. Production systems

must retrain regularly, monitor for distributional drift, and evaluate models on operational cost functions rather than symmetric metrics like F1.

Data Availability

The Elliptic Dataset is publicly available on Kaggle (search: elliptic-data-set). Code and checkpoints: <https://github.com/Saket-Maganti/gnn-fraud>.

Declaration of Competing Interest

The author declares no competing interests.

Acknowledgements

The author thanks VIT-AP University for computational resources and the Elliptic team for publicly releasing the Bitcoin fraud dataset.

References

- [1] Association of Certified Fraud Examiners, Report to the nations: 2020 global study on occupational fraud and abuse, Tech. rep., ACFE (2020).
- [2] M. Weber, J. Chen, T. Suzumura, A. Pareja, E. Law, A. Dhurandhar, C.-Y. Mavromatis, H. Kanezashi, D. Daws, S. Kumur, G. Kola, Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics, in: KDD Workshop on Anomaly Detection in Finance, 2019.
- [3] I. Alarab, S. Prakoonwit, M. I. Nacer, Competence of graph convolutional networks for anti-money laundering in bitcoin blockchain, in: Proceedings of the 2020 5th International Conference on Machine Learning Technologies, 2020, pp. 23–27.
- [4] W. W. Lo, G. K. Kulatilleke, M. Sarhan, S. Layeghy, M. Portmann, Inspection-l: Self-supervised GNN node embeddings for money laundering detection in bitcoin, Applied Intelligence 53 (2023) 19406–19417.
- [5] A. Pareja, G. Domeniconi, J. Chen, T. Ma, T. Suzumura, H. Kanezashi, T. Kaler, T. B. Schardl, C. E. Leiserson, EvolveGCN: Evolving graph convolutional networks for dynamic graphs, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 34, 2020, pp. 5363–5370.
- [6] L. Akoglu, H. Tong, D. Koutra, Graph-based anomaly detection and description: a survey, Data Mining and Knowledge Discovery 29 (2015) 626–688.
- [7] D. Wang, J. Lin, P. Cui, Q. Jia, Z. Wang, Y. Fang, Q. Yu, J. Zhou, S. Yang, Y. Qi, Semi-supervised graph attention networks for financial fraud detection, in: IEEE International Conference on Big Data, 2019, pp. 1–10.
- [8] Z. Liu, C. Chen, L. Li, J. Zhou, X. Li, L. Song, Y. Hu, GeniePath: Graph neural networks with adaptive receptive paths, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 33, 2019, pp. 4424–4431.
- [9] T. Pourhabibi, K.-L. Ong, B. H. Kam, Y. L. Boo, Fraud detection: A systematic literature review of graph-based anomaly detection approaches, Decision Support Systems 133 (2020) 113:278.

- [10] N. V. Chawla, K. W. Bowyer, L. O. Hall, W. P. Kegelmeyer, SMOTE: Synthetic minority over-sampling technique, *Journal of Artificial Intelligence Research* 16 (2002) 321–357.
- [11] T. Zhao, X. Zhang, S. Wang, GraphSMOTE: Imbalanced node classification on graphs with graph neural networks, in: *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*, 2021, pp. 833–841.
- [12] T.-Y. Lin, P. Goyal, R. Girshick, K. He, P. Dollár, Focal loss for dense object detection, in: *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 2980–2988.
- [13] A. Sankar, Y. Wu, L. Gou, W. Zhang, H. Yang, DySAT: Deep neural representation learning on dynamic graphs via self-attention networks, in: *Proceedings of the 13th International Conference on Web Search and Data Mining*, 2020, pp. 519–527.
- [14] D. Xu, C. Ruan, E. Korpeoglu, S. Kumar, K. Achan, Inductive representation learning on temporal graphs, in: *International Conference on Learning Representations*, 2020.
- [15] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, A. Bouchachia, A survey on concept drift adaptation, *ACM Computing Surveys* 46 (4) (2014) 1–37.
- [16] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, On calibration of modern neural networks, in: *Proceedings of the 34th International Conference on Machine Learning*, 2017, pp. 1321–1330.
- [17] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: *International Conference on Learning Representations*, 2017.
- [18] W. L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: *Advances in Neural Information Processing Systems*, Vol. 30, 2017.
- [19] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: *International Conference on Learning Representations*, 2018.
- [20] D. Zügner, A. Akbarnejad, S. Günnemann, Adversarial attacks on neural networks for graph data, in: *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2018, pp. 2847–2856.
- [21] C. He, K. Balasubramanian, E. Ceyani, C. Yang, H. Xie, L. Sun, et al., Fedgraphnn: A federated learning system and benchmark for graph neural networks, in: *ICLR Workshop on Distributed and Private Machine Learning*, 2021.